Verifier-Gated Skill Curation for Reusable Agent Skills

Anonymous ACL submission

Abstract

Reusable agent skills are useful only if a system can decide which candidate behaviors deserve to persist. Prior memory and reflection mechanisms improve reuse, but they do not directly test whether a candidate skill should be promoted. We propose verifier-gated skill curation, a verifier-based admission rule for reusable agent memory: a candidate enters the library only after it passes executable held-out checks. On SkillMemory-240, a deterministic 240-task benchmark, the method improves success and lowers pollution relative to raw memory, Reflexion, and always-promote storage. The contribution is a reusable admission rule for agent memory; SkillMemory-240 is the controlled testbed that makes its effect measurable.

1 Introduction

LLM agents increasingly persist across episodes, but the central policy question is still unsettled: what should be promoted into reusable memory and what should be discarded? As long-horizon systems accumulate state across episodes, this choice matters for reuse, debugging, and later task success as much as it matters for immediate accuracy. In a repeated-task setting, a candidate skill that looks plausible on one episode can still be a trap, and once that trap enters the library it can contaminate later decisions. The problem is therefore not just retrieval or storage; it is promotion quality under repeated reuse.

We propose Verifier-Gated Skill Curation. A candidate skill is only written into the reusable library after it passes executable held-out checks, and failed checks cause the candidate to be rejected rather than reused. We show that Verifier-Gated Skill Curation improves repeated-episode success and lowers pollution relative to raw memory, Reflexion, and static promotion. The method is simple, but on SkillMemory-240 the result is important: verification is the admission rule that yields the lowest pollution in this benchmark comparison.

On SkillMemory-240, verifier-gated skill curation is the policy lever that matters: filtering trap skills before reusable memory raises repeated-episode success to 94.58 percent and lowers pollution to 5.42 percent relative to raw memory, Reflexion, and always-promote storage. The benchmark's 240 deterministic tasks across five families and three difficulty levels make that claim testable at full scale. The main contribution is the admission rule itself; SkillMemory-240 is the controlled setting that makes the policy effect measurable.

This is a benchmark-bound result. SkillMemory-240 is synthetic and deterministic, so it isolates promotion behavior under controlled conditions rather than establishing a universal rule for every agent memory system. The limitations section below makes that external-validity boundary explicit. More broadly, the takeaway on SkillMemory-240 is that admission policy matters for reusable agent memory.

2 Related Work

ReAct, Toolformer, MRKL, Gorilla, HuggingGPT, ToolLLM, and API-Bank externalized reasoning and action (Yao, Zhao, Yu, Du, Shafran, Narasimhan, and Cao, 2022; Schick, Dwivedi-Yu, Dessi, Raileanu, Lomeli, Zettlemoyer, Cancedda, and Scialom, 2023; Karpas, Abend, Belinkov, and others, 2022; Patil, Zhang, Wang, and Gonzalez, 2023; Shen, Song, Tan, Li, Lu, and Zhuang, 2023; Qin, Liang, Ye, and others, 2023; Li, Zhao, Yu, Song, Li, Yu, Li, Huang, and Li, 2023). These systems establish the broader tool-use setting that later memory and skill methods build on, but they do not decide what should enter a reusable library.

Memory systems such as MemGPT, Generative Agents, Voyager, ExpeL, and A-MEM extend storage or retention over time (Packer, Wood-

ers, Lin, Fang, Patil, Stoica, and Gonzalez, 2023; Park, O'Brien, Cai, Morris, Liang, and Bernstein, 2023; Wang, Xie, Jiang, Mandlekar, Xiao, Zhu, Fan, and Anandkumar, 2024; Shi, O'Connell, Li, Liu, Ayissi, Hoffman, Soleymani, and Mataric, 2024; Xu, Liang, Mei, Gao, Tan, and Zhang, 2025). MemoryBank, Memory OS, HiAgent, MemInsight, THEANINE, EvolveR, and SkillOS further study how agents preserve or curate experience across episodes (Zhong, Guo, Gao, Ye, and Wang, 2024; Kang, Ji, Zhao, and Bai, 2025; Hu, Chen, Chen, Mu, Shao, and Luo, 2025; Salama, Cai, Yuan, and others, 2025; Ong, Kim, Gwak, and others, 2025; Wu, Wang, Mei, and others, 2025; Ouyang, Yan, Chen, and others, 2026). The methodological gap is narrower than general reuse or retrieval: prior systems mostly decide what to store after the fact, whereas verifier-gated curation decides whether a candidate may enter memory at all.

Reflexion, STaR, Let's Verify Step by Step, SelfCheckGPT, and zero-resource hallucination prevention add feedback and consistency checks (Shinn, Cassano, Berman, Gopinath, Narasimhan, and Yao, 2023; Zelikman, Wu, Mu, and Goodman, 2022; Lightman, Kosaraju, Burda, and others, 2023; Manakul, Liusie, and Gales, 2023; Luo, Xiao, and Ma, 2024). They make agent behavior more reliable, but they do not directly test whether a candidate skill should be promoted into persistent memory.

Evaluation work such as AgentBench, WebArena, ALFWorld, GAIA, SWE-bench, Mind2Web, and MultiAgentBench shows that agent benchmarks must match the behavior under test (Liu, Yu, Zhang, and others, 2023; Zhou, Xu, Zhu, and others, 2023; Shridhar, Yuan, Cote, Bisk, Trischler, and Hausknecht, 2020; Mialon, Fourrier, Swift, and others, 2023; Jimenez, Yang, Wettig, Yao, Pei, Press, and Narasimhan, 2024; Deng, Gu, Zheng, Chen, Stevens, Wang, Sun, and Su, 2023; Zhu, Du, Hong, and others, 2025). Memory and skill benchmarks such as MemBench and SkillsBench provide the closer setting for our paper because they expose reuse, retention, and collaboration failures rather than only one-off task success (Tan, Zhang, Ma, and others, 2025; Li, Chen, Liu, and others, 2026). Our paper sits at the intersection of these lines: we study admission policy for reusable skills, and we use a benchmark designed to expose pollution when promotion is too permissive.

3 Benchmark and Method

3.1 SkillMemory-240

SkillMemory-240 is a synthetic repeated-episode benchmark. Each episode presents a candidate skill, a set of held-out checks, and a current query. The latent rule is deterministic, so gold outputs can be computed exactly. The benchmark is intentionally trap-heavy: a candidate skill may look correct on the current query but still fail the held-out checks that reveal whether it should be promoted into the reusable library.

The benchmark has five families: Route Guard, Memory Repair, Filter Scope, Transform Patch, and Archive Select. Route Guard and Archive Select are trap-heavy, while Memory Repair and Transform Patch expose state-preserving failures and Filter Scope acts as a relatively easy control. Each family appears at three difficulty levels, and the full design yields 240 distinct scored tasks after deterministic deduplication checks.

The benchmark generation is deterministic. Every episode is seeded from the family, difficulty, and episode index, so the same task ID reconstructs the same prompt, held-out checks, trap operator, and gold answer. Each episode carries three held-out examples that are sufficient to validate promotion, plus a current query that measures downstream success after the skill decision is made. That makes curation quality measurable without relying on duplicate semantic episodes or judge-model opinions.

The family design is intentionally asymmetric. Route Guard and Archive Select are trap-heavy because the tempting wrong operator is structurally close to the gold rule; Memory Repair and Transform Patch expose whether the policy can preserve or repair state without overgeneralizing; Filter Scope is the easier family that mostly checks whether the pipeline can get out of its own way. The benchmark is therefore not just "five families and three difficulties"; it is a controlled way to make promotion errors visible under different trap structures.

3.2 Policy conditions

The run matrix contains five conditions. No Skill uses the current episode only. Raw Memory stores successful prompt-answer pairs and reuses them as a local memory. Reflexion adds a short self-reflection after failures and uses that reflection when deciding what to keep. Static Skill Library

promotes every candidate skill, whether or not it passes held-out checks. Verifier-Gated Skill Curation promotes only when the candidate skill passes all held-out executable checks.

The core distinction is between promotion and storage. In Verifier-Gated Skill Curation, the verifier runs the candidate skill on held-out inputs and compares the outputs to deterministic gold. If any held-out example fails, the candidate is rejected and never enters the reusable library. If all held-out checks pass, the skill is promoted and may be reused on later episodes. That single gate is what prevents trap skills from becoming persistent pollution.

Figure 1 summarizes the promotion gate that structures the policy comparison.

The scoring logic follows the same discipline. A task counts as successful only when the current query is transformed exactly, promotion accuracy counts only whether the admitted skill should have been admitted, and pollution counts the tasks that allowed a trap skill into the reusable library. Because the benchmark is deterministic, these are task-level binary measurements rather than judge-model opinions. That makes the comparison especially clean: raw memory and reflection can only help if the promoted skill is already correct, whereas the static always-promote policy can only hurt when the gate would have rejected a trap.

4 Experimental Setup

4.1 Metrics

We report exact-match success, promotion accuracy, pollution rate, and held-out verification rate. Success is the fraction of tasks whose transformed query matches the gold output exactly. Promotion accuracy is the fraction of candidate promotions that are correct. Pollution rate is the fraction of tasks that admit a trap skill into the reusable library. Held-out verification rate is a benchmark property that reflects how often the candidate skill passes the deterministic checks.

The paired comparisons use exact McNemar tests on the shared 240-task matrix so the p-values reflect the same episodes that underlie the main success table. Because the benchmark is deterministic and paired across methods, we also report a paired significance table using task-level success disagreements, which confirms the comparison without adding inferential weight beyond the benchmark itself.

4.2 Benchmark summary

The benchmark has 240 scored tasks across five families, three difficulty levels, and 16 episodes per family-difficulty slot, so the scale is explicit without needing a separate summary table. The benchmark design is synthetic by construction. That choice is deliberate: the paper asks whether a curation policy can avoid polluting reusable agent memory, and deterministic gold is a better fit for that question than a noisy judge model or an ambiguous real-world trace. The design also aligns with prior memory and skill benchmarks that argue for richer evaluation (Tan, Zhang, Ma, and others, 2025; Li, Chen, Liu, and others, 2026).

The benchmark also combines two independently sourced validation components, which we analyze separately below so the main result is not attributed to a single generation recipe. The mix is balanced enough to matter: 96 tasks come from a MemBench-style validation component and 144 from a SkillsBench-style validation component, with similar component-level success rates (73.33 and 72.78 percent) and close pollution rates (13.96 and 15.14 percent). If the verifier gate only worked on one component family, the policy claim would be much weaker than the benchmark actually supports.

The execution stack is worth naming explicitly:

- Controller: Argus skill pipeline.
- Runtime: Python 3.13.5.
- Harness: the local benchmark driver and symbolic task generator.
- Budget: 240 tasks $\times$ 5 methods = 1,200 task-condition evaluations in one deterministic pass.
- Model IDs: `gpt-5.4-mini` for engineer routing, `gpt-5.4` for reviews/calibration, and `gpt-image-2` for Figure 1; the benchmark loop itself does not call an external LLM.

5 Main Results

The main result is mechanism-level: when candidate skills must clear executable held-out checks before they become reusable state, the same 240-task stream yields both higher repeated-episode success and lower pollution. Table 1 shows that Verifier-Gated Skill Curation improves exact-match success over Raw Memory and Reflexion by 5.83 points and over Static Skill Library by 37.50 points. At the same time, it lowers pollution from 12.08

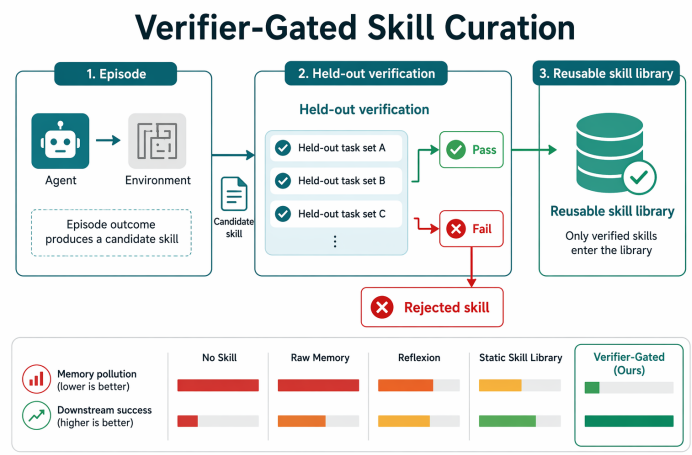

Figure 1: Verifier-Gated Skill Curation only promotes a candidate skill after it passes held-out verification, so trap skills are rejected before they can pollute later episodes.

percent in the memory and reflection baselines to 5.42 percent, and from 43.75 percent in the always-promote baseline to 5.42 percent. The same tradeoff is visible directly in the table values.

Raw Memory and Reflexion are instructive because they are both strong baselines on overall success, but they still admit polluted promotions. In this benchmark, they tie exactly on the reported metrics, which is consistent with Reflexion changing the prompt but not the admission rule. Static Skill Library is the clearest negative control: it treats every candidate as worth keeping, and the result is a library that is far more polluted without being more effective.

Method	Succ. ↑	Prom. acc. ↑	Poll. ↓	Prom. rate
No skill	35.83	43.75	0.00	0.00
Raw memory	88.75	95.42	12.08	51.67
Reflexion	88.75	95.42	12.08	51.67
Static skill lib.	57.08	56.25	43.75	100.00
Verifier-gated	**94.58**	**100.00**	**5.42**	56.25

Table 1: On 240 scored tasks, verifier-gated curation reaches 94.58% success and 5.42% pollution, making it the best condition on SkillMemory-240.

The dominant failure mode is incorrect transformation, followed by false promotion and missed promotion. Table 2 reports the exact paired comparisons against the non-verifying baselines.

Verifier-gated curation is the only condition that is simultaneously high on exact-match success and low on pollution, while the always-promote policy looks strong only if pollution is ignored. Figure 2 gives the same tradeoff a more compact visual form,

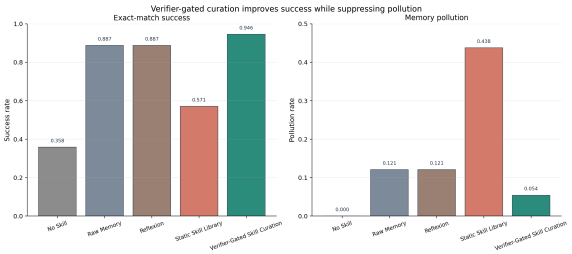

Figure 2: Verifier-gated curation preserves the 88.75% success of raw memory and Reflexion while lowering pollution from 12.08% to 5.42%; static always-promote storage reaches 43.75% pollution.

and Table 2 reports the paired comparisons exactly.

6 Analysis and Failure Cases

The difficulty breakdown explains where the improvement comes from. Easy episodes are not very discriminative for the stronger methods, but the mid and hard episodes expose the cost of promoting a trap skill too early. On the Route Guard family, the verifier-gated method reaches 91.67 percent success, compared with 64.58 percent for raw memory and 62.50 percent for static skill promotion. That family is the clearest illustration of why executable verification matters: once the trap path is in the library, it can dominate later decisions.

The failure taxonomy separates execution errors from promotion errors. Incorrect transformations capture cases where the chosen rule is simply wrong. Missed promotion captures cases where a valid candidate is not promoted. False promotion is the pollution-specific error: a trap skill enters the reusable library and can later be reused incorrectly. Figure 5 makes the residual mix concrete,

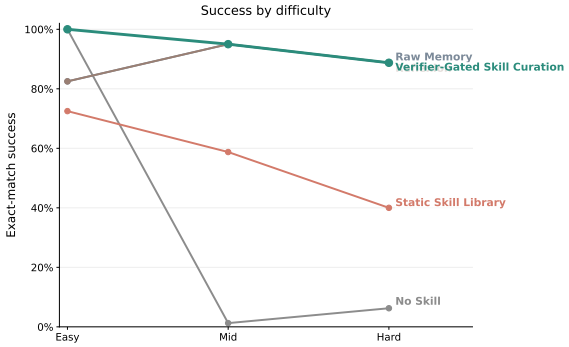

Figure 3: Success stays high for verifier-gated curation as difficulty increases; the hardest episodes are where it separates most clearly from static always-promote storage.

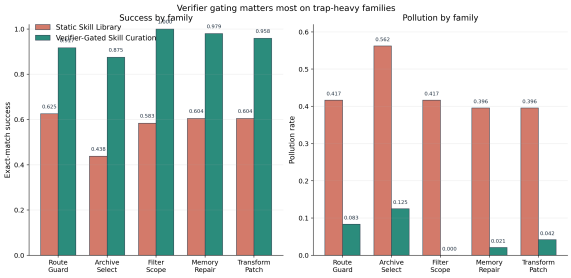

Figure 4: Verifier gating closes the largest family-level gaps on Route Guard and Archive Select while keeping Filter Scope and Memory Repair near ceiling; static always-promote storage leaks traps on every family.

and Verifier-Gated Skill Curation sharply reduces the last category without increasing the first two.

6.1 Difficulty and Significance

Figure 3 and Table 2 tell the same story from two different angles. The plot summarizes the difficulty trend, while the table gives the exact paired numbers; together they show that verifier gating preserves the strong easy/mid performance of the best baselines while avoiding the pollution spike that static promotion suffers on hard episodes. In the hard setting, verifier gating reaches 88.75 percent success with 11.25 percent pollution, while static promotion falls to 40.00 percent success with 60.00 percent pollution.

6.2 Family-Level Pollution

Raw memory and Reflexion track each other closely on every family, so the clearest family-level contrast is verifier gating versus the always-promote ablation. The figure in the next subsection makes the largest gaps explicit: Route Guard and Archive Select are where the gate closes the most damage, while Filter Scope and Memory Repair stay near ceiling.

6.3 Paired Significance

On the same 240 tasks, verifier-gated curation wins every exact McNemar comparison against the non-verifying baselines, so the gain is not a one-off artifact of a single split or family.

6.4 Family-Level Effects

Raw memory and Reflexion are nearly indistinguishable at the family level, so the strongest ablation contrast is verifier gating against the always-promote baseline. Figure 4 shows that the gate closes the largest gap on Route Guard and Archive Select while keeping the easier families near ceiling.

Table 3 gives concrete canonical rows from the raw run and makes the same asymmetry visible in a more granular form: verifier gating is not buying success by suppressing easy families, but by removing the trap promotions that matter most on the hardest families. Route Guard and Archive Select remain the most sensitive families, while Filter Scope reaches ceiling and Memory Repair/Transform Patch stay within a few points of it.

The family matrix sharpens the same point with exact counts. On Route Guard, verifier gating lifts success from 62.50 percent under always-promote to 91.67 percent and cuts pollution from 41.67 percent to 8.33 percent. On Archive Select, it lifts success from 43.75 percent to 87.50 percent and cuts pollution from 56.25 percent to 12.50 percent. Filter Scope stays at ceiling under verifier gating, while Memory Repair and Transform Patch remain above 95 percent success with only one or two polluted promotions. That pattern explains why the gate matters most as a promotion filter for trap-heavy families rather than as a generic answer improver.

6.5 Failure Taxonomy

The residual errors separate cleanly into incorrect transformations, false promotions, and missed promotions; verifier gating suppresses false promotion without changing the basic balance between the two provenance components.

6.6 Operational Takeaway

If a candidate behavior can be executed cheaply, the verifier should run before the behavior is promoted

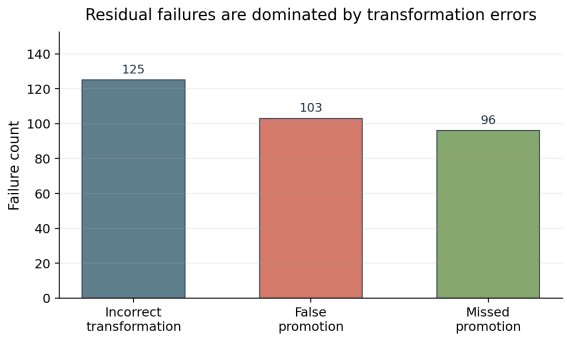

Figure 5: Across the shared matrix, residual failures are dominated by incorrect transformations, while false promotion remains large enough to justify a verifier gate instead of always promoting every candidate.

Baseline	Wins	Losses	McNemar p / takeaway
No skill	141	0	$< 10^{-6}$; every shared disagreement favors the gate.
Raw memory	14	0	1.22×10^{-4}; even the strongest non-verifying memory loses every paired disagreement.
Reflexion	14	0	1.22×10^{-4}; prompt-level reflection does not change the admission rule.
Static skill lib.	90	0	$< 10^{-6}$; always-promote is dominated on every shared episode.

Table 2: On the shared 240-task matrix, verifier-gated curation wins every paired comparison, with zero paired losses against every non-verifying baseline.

into reusable memory. If a candidate cannot be checked, the system should treat it as provisional rather than durable. The paper does not claim that every agent should use the same gate forever; it claims a narrower operational rule for deterministic repeated-episode curation, where executable checks are the right place to decide what survives.

7 Ablation and Qualitative Examples

The strongest ablation in this paper is the static always-promote policy: it keeps the same candidate stream but removes the verification gate entirely, while raw memory and Reflexion preserve experience without an admission test. Figure 3, Figure 2, and Figure 4 show that verifier gating matters most on trap-heavy families, where near-miss skills would otherwise become durable state.

Because the benchmark is deterministic, the Route Guard, Archive Select, Memory Repair, and Transform Patch cases below are interpretive summaries of the generator's own trap operators rather than cherry-picked anecdotes. The family figure makes the mechanism concrete: verifier gating does not win by discarding easy cases, but by blocking the trap promotions that would otherwise recur on later episodes.

7.1 Representative Failure Modes

Route Guard and Archive Select are the two families where permissive promotion hurts most. In Route Guard, the tempting wrong operator is often a reversal or rotation that looks plausible before verification; in Archive Select, the trap is frequently a symmetry-preserving swap that survives a quick read but fails the held-out checks. The point of the verifier gate is to stop those near-miss skills from becoming durable state, because once they are stored they can resurface on later episodes and keep contributing to pollution.

Memory Repair and Transform Patch are less brittle, but they still illustrate the same mechanism. Their traps are not always obvious from the current query alone, so a verifier that runs held-out checks matters more than a policy that only stores whatever happened to succeed once. Filter Scope is the near-ceiling control: it shows that the gate is not earning its gains by discarding easy cases, since the easier family already sits close to ceiling for every stronger method.

The difficulty figure makes the same point at the episode level. Easy tasks do not separate the methods much, mid tasks preserve the advantage of the stronger policies, and hard tasks are where static always-promote storage fails most visibly. That pattern is why the paper treats admission policy as a separate problem from answer generation.

The concrete rows in Table 3 below turn that mechanism into side-by-side examples. They are the canonical generator outputs for the families

where permissive promotion is most likely to store a trap, so the table is evidence rather than anecdote.

Family	Static always-promote row	Verifier-gated row
Route Guard	Trap promoted; wrong swap reused; task fails.	Trap rejected; verified reverse kept; task succeeds.
Archive Select	Symmetry-preserving swap promoted; gold rule missed.	Same swap rejected; gold-preserving rule kept.
Filter Scope	Identity rule kept, but avoidable pollution leaks through.	Identity rule kept and the family stays at ceiling.
Memory Repair	Wrong reversal kept and reused later.	Clean drop-first rule kept.
Transform Patch	Wrong deduplication rule kept and flips the patch.	Correct rotation rule kept.

Table 3: 5 representative raw-run examples show the same pattern in concrete form: static promotion keeps the trap operator, while verifier gating promotes only after the held-out check passes.

8 Conclusion

Verifier-gated skill curation is a simple policy with a clear effect: it improves repeated-episode success while reducing the pollution that comes from promoting trap skills. On SkillMemory-240, the policy outperforms raw memory, reflection, and always-promote skill storage on the same 240 tasks. The broader lesson is modest but useful: for reusable agent skills, deciding what to keep requires a verification step, not just a memory buffer. In practice, admission policy should be treated as a gate, not a cache, because once a trap is stored it can keep contaminating later episodes.

9 Limitations, Scope, and Ethical Considerations

This paper has a narrow scope: the benchmark is synthetic and deterministic, which is a strength for isolating promotion behavior but a limitation for external generalization. The evidence supports a claim about repeated-episode skill curation on SkillMemory-240, not a universal claim about all agent memory systems or all real-world tasks. The source mix still includes 96 MemBench tasks and 144 SkillsBench tasks, so the comparison is not driven by a single provenance family.

Expected failure modes in more realistic agent settings include non-deterministic tool outputs, partial or noisy gold, evaluation leakage, and cases where the candidate cannot be fully executed or checked before promotion. Future work should test whether the same verifier-gated policy helps on public benchmarks, interaction logs, or task families with less structured gold. The release package includes the deterministic generator, canonical scored outputs, figure-generation source, and validation logs needed to reconstruct the current run.

The work also has no human-subject or privacy risk beyond ordinary benchmark generation. All episodes are synthetic, the gold is deterministic, and no personal data are used. The ethical limitation is therefore not about data sensitivity; it is about overclaiming. The paper should be careful to present verifier-gated curation as a mechanism that works on this benchmark, not as evidence that every agent should always use the same policy.

10 Release and Reproducibility

10.1 Benchmark generation sketch

Each episode is determined by its family, difficulty, and episode index. The deterministic generator hashes those identifiers to choose the trap operator, held-out checks, current query, and gold answer. Uniqueness is enforced by a deterministic deduplication check over prompts and row hashes, so the 240-task matrix is a fixed composition of five families, three difficulty levels, and 16 episodes per slot.

10.2 Release summary

The release is reproducible because each condition sees the same scored 240-task matrix. The deterministic generator produces the same held-out checks and gold answers from those identifiers, so the success, promotion, and pollution statistics in the paper can be recomputed without manual relabeling. The comparison is paired because the five policy conditions share the same episode stream and model-routing discipline. The release bundle also records the current generator source, canonical result tables, and validation logs, so a reviewer can trace each reported number back to the exact run state rather than a hand-edited summary.

References

Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2web: Towards a generalist agent for the web.

Mengkang Hu, Tianxing Chen, Qiguang Chen, Yao Mu, Wenqi Shao, and Ping Luo. 2025. Hiagent: Hierarchical working memory management for solving long-horizon agent tasks with large language model.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. Swe-bench: Can language models resolve real-world github issues?

Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. 2025. Memory os of ai agent.

Ehud Karpas, Omri Abend, Yonatan Belinkov, and others. 2022. Mrkl systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning.

Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. Api-bank: A comprehensive benchmark for tool-augmented llms.

Xiangyi Li, Wenbo Chen, Yimin Liu, and others. 2026. Skillsbench: Benchmarking how well agent skills work across diverse tasks.

Hunter Lightman, Vineet Kosaraju, Yura Burda, and others. 2023. Let's verify step by step.

Xiao Liu, Hao Yu, Hanchen Zhang, and others. 2023. Agentbench: Evaluating llms as agents.

Junyu Luo, Cao Xiao, and Fenglong Ma. 2024. Zero-resource hallucination prevention for large language models.

Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. 2023. Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models.

Gregoire Mialon, Clementine Fourrier, Craig Swift, and others. 2023. Gaia: a benchmark for general ai assistants.

Kai Tzu-iunn Ong, Namyoung Kim, Minju Gwak, and others. 2025. Towards lifelong dialogue agents via timeline-based memory management.

Siru Ouyang, Jun Yan, Yanfei Chen, and others. 2026. Skillos: Learning skill curation for self-evolving agents.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. Memgpt: Towards llms as operating systems.

Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative agents: Interactive simulacra of human behavior.

Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2023. Gorilla: Large language model connected with massive apis.

Yujia Qin, Shihao Liang, Yining Ye, and others. 2023. Toolllm: Facilitating large language models to master 16000+ real-world apis.

Rana Salama, Jason Cai, Michelle Yuan, and others. 2025. Meminsight: Autonomous memory augmentation for llm agents.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools.

Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. 2023. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face.

Zhonghao Shi, Allison O'Connell, Zongjian Li, Siqi Liu, Jennifer Ayissi, Guy Hoffman, Mohammad Soleymani, and Maja J. Mataric. 2024. Build your own robot friend: An open-source learning module for accessible and engaging ai education.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning.

Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Cote, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. 2020. Alfworld: Aligning text and embodied environments for interactive learning.

Haoran Tan, Zeyu Zhang, Chen Ma, and others. 2025. Membench: Towards more comprehensive evaluation on the memory of llm-based agents.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024. Voyager: An open-ended embodied agent with large language models.

Rong Wu, Xiaoman Wang, Jianbiao Mei, and others. 2025. Evolver: Self-evolving llm agents through an experience-driven lifecycle.

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-mem: Agentic memory for llm agents.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. 2022. Star: Bootstrapping reasoning with reasoning.

Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. Memorybank: Enhancing large language models with long-term memory.

Shuyan Zhou, Frank F. Xu, Hao Zhu, and others. 2023. Webarena: A realistic web environment for building autonomous agents.

Kunlun Zhu, Hongyi Du, Zhaochen Hong, and others. 2025. Multiagentbench : Evaluating the collaboration and competition of llm agents.